

3. Given the sample order form and description below, create the un-normalized form of the data and transform through the steps needed to move into normalized form.

The Gill Art Gallery wishes to maintain data on their customers, artists and paintings. They may have several paintings by each artist in the gallery at one time. Paintings may be bought and sold several times. In other words, the gallery may sell a painting, then buy it back at a later date and sell it to another customer.

Ans (3): Unnormalized Form (UNF): The initial relation combines customer details with all purchased paintings into a single order form, resulting in multi-valued fields and repeating groups.

The relation suffers from data redundancy and update anomalies due to repeating groups created when a customer buys multiple paintings. Specifically, customer details recur with each purchase, artist information is duplicated for every artwork they produce, and painting records repeat across multiple sales.

First Normal Form (1NF): The relation is converted into 1NF by ensuring that:

- Each field is allowed only atomic (single-valued) data for each field.
- Each row represents one painting purchase.
- no repeating groups is to be allowed.

Art Gallery
customerName
Address,
Phone
artistID
artistName
paintingTitle
PurchaseDate
sales

The repeating purchase records are separated into individual rows. Every attribute now contains a single value, satisfying the requirements of First Normal Form.

Second Normal Form (2NF): To achieve **Second Normal Form (2NF)**, partial dependencies are removed.

The original data contains information about three different entities:

- Customer
- Artist
- Painting
- Sale

Therefore, separate relations are created.

The following should be the tables and their respective attributes:

tables	Attributes	Primary key	Foreign key
customer	CustomerID, CustomerName, Address, phone	CustomerID	-
Artist	artistID,ArtistName	ArtistID	-
painting	paintingID,title,artistID	paintingID	artistID
sale	saleID, customerID, paintingID, purchasedate, salesPrice	saleID	CustomerID,PaintingID

The following attributes were separated into their respective relations:

CUSTOMER

- CustomerName
- Address
- Phone

These attributes are only to be depended on **CustomerID**.

ARTIST

- ArtistName

This attribute depends only on **ArtistID**.

PAINTING

- Title

This attribute depends only on **PaintingID**.

The **SALE** table stores only the transaction details:

- PurchaseDate
- SalesPrice

along with the foreign keys identifying the customer and the painting involved in the sale.

This decomposition removes partial dependencies and reduces data redundancy.

Third Normal Form (3NF): The relations obtained in 2NF already satisfy the conditions of Third Normal Form because there are no transitive dependencies. Every non-key attribute depends only on the primary key of its respective relation.

The normalization process removes redundancy by storing each type of information only once.

- **Customer information** is stored only in the **CUSTOMER** table instead of being repeated for every purchase.
- **Artist information** is stored only in the **ARTIST** table regardless of how many paintings the artist has created.
- **Painting information** is stored only in the **PAINTING** table, while each purchase or resale of a painting is recorded as a separate transaction in the **SALE** table.

This design satisfies the following business rules:

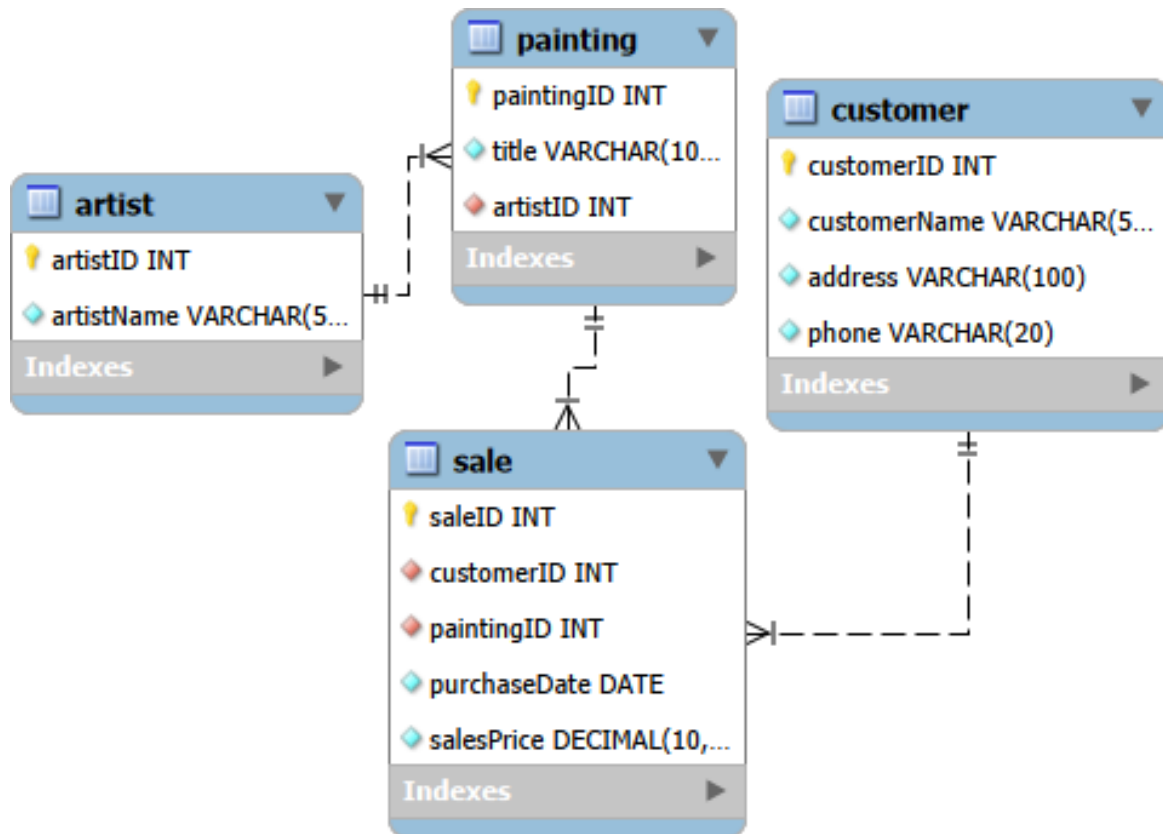
- 1 artist can create N paintings.
- 1 customer can purchase N paintings.
- A painting can be sold M or N times after the gallery buys it back again.

By removing partial and transitive dependencies, the normalized design prevents data redundancy and guarantees consistency, successfully bringing the database into Third Normal Form (3NF).

3(a) Design a relational database. Create a relational schema which should include the following information:

➔ Relation name, Attributes, Primary Key, Foreign Keys

Ans 3(a): EER Diagram (Relational Schema Representation)



3(b) Write SQL CREATE TABLE statements for creating tables including: Primary Keys, Foreign Keys, Appropriate data types, NOT NULL constraints

SOURCE CODE:

```
create database lab7third;
```

```
use lab7third;
```

```
create table artist(  
artistID int primary key,  
artistName varchar(40) not null  
);
```

```
Describe artist;
```

	Field	Type	Null	Key	Default
►	artistID	int	NO	PRI	NULL
	artistName	varchar(50)	NO		NULL

```
create table customer(  
customerID int primary key,  
customerName varchar(40) not null,  
address varchar(60) not null,  
phone varchar(15) not null  
); describe customers;
```

	Field	Type	Null	Key	Default
►	customerID	int	NO	PRI	NULL
	customerName	varchar(50)	NO		NULL
	address	varchar(100)	NO		NULL
	phone	varchar(20)	NO		NULL

```
create table painting(  
paintingID int primary key,  
title varchar(30) not null,  
artistID int not null,
```

	Field	Type	Null	Key	Default
►	paintingID	int	NO	PRI	NULL
	title	varchar(100)	NO		NULL
	artistID	int	NO	MUL	NULL

```
foreign key (artistID) references artist(artistID)  
); describe painting;
```

```
create table sale(  
saleID int primary key,  
customerID int not null,  
paintingID int not null,  
purchaseDate date not null,  
salesPrice decimal(12, 2) not null,  
foreign key (customerID) references customer(customerID),  
foreign key (paintingID) references painting(paintingID)  
); describe sale ;
```

	Field	Type	Null	Key	Default
►	saleID	int	NO	PRI	NULL
	customerID	int	NO	MUL	NULL
	paintingID	int	NO	MUL	NULL
	purchaseDate	date	NO		NULL
	salesPrice	decimal(10,2)	NO		NULL